Chapter 8: Strings

Strings are sequence of characters. In C, there is no built-in data type for strings. String can be represented as a one-dimensional array of characters. A character string is stored in an array of character type, one ASCII character per location. String should always have a NULL character (‘\0’) at the end, to represent the end of string.

String constants can be assigned to character array variables. String constants are always enclosed within double quotes and character constants are enclosed within single quotes.

READING STRINGS (COMPILE TIME)

Examples:

(1) char c[4]={‘s’,’u’,’m’,’\0’);

(2) char str[16]="qwerty";

Creates a string. The value at str[5] is the character ‘y’. The value at str[6] is the null character. The values from str[7] to str[15] are undefined.

(3)
```
char name[5];
int main( )
{
    name[0] = ‘G’;
    name[1] = ‘O’;
    name[2] = ‘O’;
    name[3] = ‘D’;
    name[4] = ‘\0’;
    return 0;
}
```

(4) char name[5] = “INDIA”

Strings are terminated by the null character, it is preferred to allocate one extra space to store null terminator.

String can be read either character-by-character or as an entire string (using %s format specifier). Array name itself specifies the base address and %s is a format specifier which will read a string until a white space character is encountered.

[Note: No need to use & operator while reading string using %s]

RUN TIME INITIALIZATION AND PRINTING

Example:

```
char name[20];
int i=0;
while((name[i] = getchar ()) != '\n' )
        i++;
scanf( "%s" , name);
printf("%s" , name);
```

STRING MANIPULATION FUNCTIONS

C does not provide any operator, which manipulates the entire string at once. Strings are manipulated either using pointers or special routines (built-in) available from the standard string library **string.h.**

The following is the list of string functions available in string.h:

strcpy(string1, string2)	Copy string2 into string1
strcat(string1, string2)	Concatenate string2 onto the end of string1
strcmp(string1, string2)	Lexically compares the two input strings (ASCII comparison) returns 0 if string1 is equal to string2 < 0 if string1 is less than string2 > 0 if string1 is greater than string2
strlen (string)	Gives the length of a string
strrev (string)	Reverse the string and result is stored in same string.
strncat(string1, string2, n)	Append n characters from string2 to string1
strncmp(string1, string2, n)	Compare first n characters of two strings.
strncpy(string1,string2, n)	Copy first n characters of string2 to string1
strupr (string)	Converts string to uppercase
strlwr (string)	Converts a string to lowercase
strchr (string, c)	Find first occurrence of character c in string.
strrchr (string, c)	Find last occurrence of character c in string.

Examples

---

1. strlen

strlen(s1) calculates the length of string s1.

```
#include <stdio.h>
#include <string.h>
int main()
{
  char name[ ]= "Hello";
  int len1, len2;
  len1 = strlen(name);
  len2 = strlen("Hello World");
  printf("length of %s = %d\n", name, len1);
  printf("length of %s = %d\n", "Hello World", len2);
  return 0;
}
```

Output

```
length of Hello = 5
length of Hello World = 11
strlen doesn't count '\0' while calculating the length of a string.
```

2. strcat

strcat(s1, s2) concatenates(joins) the second string s2 to the first string s1.

```
#include <stdio.h>
#include <string.h>
int main()
{
  char s2[ ]= "World";
  char s1[20]= "Hello";
```

```
  strcat(s1, s2);
  printf("Source string = %s\n", s2);
  printf("Target string = %s\n", s1);
  return 0;
}
```

Output

Source string = World
Target string = HelloWorld

3. strncat

strncat(s1, s2, n) concatenates(joins) the first ‘n’ characters of the second string s2 to the first string s1.

```
#include <stdio.h>
#include <string.h>
int main()
{
  char s2[ ]= "World";
  char s1[20]= "Hello";
  strncat(s1, s2, 2);
  printf("Source string = %s\n", s2);
  printf("Target string = %s\n", s1);
  return 0;
}
```

Source string = World
Target string = HelloWo

4. strcpy

strcpy(s1, s2) copies the second string s2 to the first string s1.

```
#include <string.h>
#include <stdio.h>
int main()
{
```

```
  char s2[ ]= "Hello";
  char s1[10];
  strcpy(s1, s2);
  printf("Source string = %s\n", s2);
  printf("Target string = %s\n", s1);
  return 0;
}
```

Output

Source string = Hello
Target string = Hello

5. strncpy

strncpy(s1, s2, n) copies the first 'n' characters of the second string s2 to the first string s1.

```
#include <string.h>
#include <stdio.h>
int main()
{
  char s2[ ]= "Hello";
  char s1[10];
  strncpy(s1, s2, 2);
  s1[2] = '\0';  /* null character manually added */
  printf("Source string = %s\n", s2);
  printf("Target string = %s\n", s1);
  return 0;
}
```

Output

Source string = Hello
Target string = He
Please note that we have added the null character ('\0') manually.

6. strcmp

strcmp(s1, s2) compares two strings and finds out whether they are same or different. It compares the two strings character by character till there is a mismatch. If the two strings

are **identical**, it returns a **0**. If not, then it returns the difference between the ASCII values of the first non-matching pair of characters.

```
#include <stdio.h>
#include <string.h>
int main()
{
  char s1[ ]= "Hello";
  char s2[ ]= "World";
  int i, j;
  i = strcmp(s1, "Hello");
  j = strcmp(s1, s2);
  printf("%d \n %d\n", i, j);
  return 0;
}
```

0
-15
The difference between the ASCII values of H(72) and W(87) is -15.

7. strncmp

strncmp(s1, s2, n) compares the first 'n' characters of s1 and s2.

```
#include <stdio.h>
#include <string.h>
int main()
{
  char s1[ ]= "Hello";
  char s2[ ]= "World";
  int i, j;
  i = strncmp(s1, "He BlogsDope", 2);
  printf("%d\n", i);
  return 0;
}
```

Output
0

8. strchr

strchr(s1, c) returns a pointer to the first occurrence of the character c in the string s1 and returns NULL if the character c is not found in the string s1.

```
#include <stdio.h>
#include <string.h>
int main()
{
  char s1[ ]= "Hello Blogsdope";
  char c = 'B';
  char *p;
  p = strchr(s1, c);
  printf("%s\n", p);
  return 0;
}
```

Output

Blogsdope

9. strrchr

strrchr(s1, c) returns a pointer to the last occurrence of the character c in the string s1 and returns NULL if the character c is not found in the string s1.

```
#include <stdio.h>
#include <string.h>
int main()
{
  char s1[ ]= "Hello Blogsdope";
  char c = 'o';
  char *p;
  p = strrchr(s1, c);
  printf("%s\n", p);
  return 0;
}
```

Output

ope

10. strlwr()

strlwr( ) function converts a given string into lowercase.

```
#include<stdio.h>
#include<string.h>
int main()
{
   char str[ ] = "MODIFY This String To LOwer";
   printf("%s\n",strlwr (str));
   return  0;
}
```

Output:

```
modify this string to lower
```

11. strupr()

strupr( ) function converts a given string into uppercase.

```
#include<stdio.h>
#include<string.h>
int main()
{
   char str[ ] = "Modify This String To Upper";
   printf("%s\n",strupr(str));
   return  0;
}
```

Output:

```
MODIFY THIS STRING TO UPPER
```

12. strrev()

strrev( ) function reverses a given string in C language.

```
#include<stdio.h>
#include<string.h>
int main()
{
  char name[30] = "Hello";
  printf("String before strrev( ) : %s\n",name);
  printf("String after strrev( )  : %s",strrev(name));
  return 0;
}
```

Output:

```
String before strrev( ) : Hello
String after strrev( )     : olleH
```

Write programs to perform manipulation of strings without using string inbuilt function:

1. Calculate String length:

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.    char s[1000];
5.    int c = 0;
6.    printf("Input a string\n");
7.    gets(s);
8.    while (s[c] != '\0')
9.      c++;
10.   printf("Length of the string: %d\n", c);
11.   return 0;
12. }
13.
14.
```

OR using for loop

```
#include <stdio.h>
int main()
```

```
{
   char s[1000];
   int i;
   printf("Enter a string: ");
   scanf("%s", s);
   for(i = 0; s[i] != '\0'; ++i);
   printf("Length of string: %d", i);
   return 0;
}
```

2. C Program to Copy String Without Using strcpy()

```
#include <stdio.h>
int main()
{
   char s1[100], s2[100], i;
   printf("Enter string s1: ");
   scanf("%s",s1);                  // gets(s1); can also be used
   for(i = 0; s1[i] != '\0'; ++i)
   {
      s2[i] = s1[i];
   }
   s2[i] = '\0';
   printf("String s2: %s", s2);
   return 0;
}
```

3. C Program to Compare Two Strings Without Using Library Function strcmp()

```
1  #include<stdio.h>
2
3  int main() {
4    char str1[30], str2[30];
5    int i;
6    printf("\nEnter two strings :");
7    gets(str1);
```

```
8     gets(str2);
9     i = 0;
10    while (str1[i] == str2[i] && str1[i] != '\0')
11       i++;
12    if (str1[i] > str2[i])
13       printf("str1 > str2");
14    else if (str1[i] < str2[i])
15       printf("str1 < str2");
16    else
17       printf("str1 = str2");
18    return (0)
19 }
20
21
22
```

OR

```
#include<stdio.h>
int main()
{
      char string1[5],string2[5];
      int i,temp = 0;
      printf("Enter the string1 value: ");
      gets(string1);
      printf(" Enter the String2 value: ");
      gets(string2);
      for(i=0; string1[i]!='\0'; i++)
      {
            if(string1[i] != string2[i])
                        {
                         temp = 1;
                         break;
                        }
```

```
    }
    if(temp == 1)
    printf("Both strings are same.");
    else
    printf("Both string not same.");
    return 0;
}
```

4. C program for string concatenation without using strcat()

```
/* C program to concatenate two strings without
 * using standard library function strcat()
 */
#include <stdio.h>
int main()
{
  char str1[50], str2[50], i, j;
  printf("\nEnter first string: ");
  scanf("%s",str1);
  printf("\nEnter second string: ");
  scanf("%s",str2);

  /* This loop is to store the length of str1 in i
   * It just counts the number of characters in str1
   * You can also use strlen instead of this.
   */
  for(i=0; str1[i]!='\0'; ++i);
  /* This loop would concatenate the string str2 at
   * the end of str1
   */
  for(j=0; str2[j]!='\0'; ++j, ++i)
  {
    str1[i]=str2[j];
  }
  // \0 represents end of string
  str1[i]='\0';
```

```
  printf("\nOutput: %s",str1);
  return 0;
}
```

5. C Program to Reverse string without using strrev

```
#include <stdio.h>
#include <string.h>
int main()
{
char mystring[50];
int len, i;

//print a string about the program
printf("C Program to reverse a string\n");
printf("Enter a string: ");
//get the input string from the user
scanf("%s", mystring);
//find the length of the string using strlen()
len = strlen(mystring);
//loop through the string and print it backwards
for(i=len-1; i>=0; i--)
{
printf("%c", mystring[i]);
}
   return 0;
}
```

OR using for loop

```
#include<stdio.h>
#include<string.h>
int main()
{
  char str[100], temp;
  int i, j = 0;
```

```
  printf("\nEnter the string :");
  gets(str);

  i = 0;
  j = strlen(str) - 1;

  while (i < j)
  {
    temp = str[i];
    str[i] = str[j];
    str[j] = temp;
    i++;
    j--;
  }
  printf("\nReverse string is :%s", str);
  return (0);
}
```